Lab 5: Search Optimization and Troubleshooting

Lab Overview

Every search you have written so far returned the right answer. This lab is about making searches return the right answer *efficiently*, and about diagnosing them when they do not behave as expected.

You will measure the effect of each search best practice against the course dataset, read the Job Inspector to see what Splunk actually did, and repair three searches that are written badly on purpose.

Learning Objectives

By completing this lab, you will:

- Use the time range as your primary filter and measure the difference
- Scope searches with `index` and `sourcetype` instead of bare keywords
- Choose inclusion over exclusion when filtering
- Filter early in the pipeline rather than after a transforming command
- Place wildcards so Splunk can still use its indexes
- Read the Job Inspector to compare events scanned against events matched
- Diagnose and repair searches that return the wrong results

Prerequisites

- A running Splunk instance with the course data loaded (completed Lab 1)
- Basic search skills (completed Lab 2)
- Field usage skills (completed Lab 3)
- Familiarity with `table`, `sort` and `dedup` (completed Lab 4)

> **Time range**: Unless a step says otherwise, use **Last 30 days**. Several exercises deliberately change the time range so you can observe the effect.

Exercise 1: Time Is the Most Efficient Filter

Overview

Splunk stores events in time-ordered buckets. Narrowing the time range lets it skip whole buckets without reading them. No other filter is as cheap.

Task 1.1: Measure a Wide Time Range

Set the time picker to **Last 30 days** and run:

```
index=main sourcetype=access_combined_wcookie
| stats count
```

Record the event count.

Task 1.2: Measure a Narrow Time Range

Change the time picker to **Last 24 hours** and run the same search:

```
index=main sourcetype=access_combined_wcookie
| stats count
```

✓ **Checkpoint**: Record both counts. The narrower range returns far fewer events because Splunk never opened the older buckets.

Task 1.3: Compare Against All Time

Set the time picker to **All time** and run the search once more. Note that the count matches Last 30 days — the dataset only spans about 26 days — but the search had to confirm that by checking every bucket.

✓ **Checkpoint**: Same answer, more work. A time range is not only about correctness; it is about how much data Splunk must rule out.

Exercise 2: Specify Index and Source Type

Overview

A bare search term makes Splunk look across every index it can read. Naming the index and source type narrows the search before any matching begins.

Task 2.1: Search Without Scope

```
purchase
```

This searches all accessible indexes for the term.

Task 2.2: Add the Index

```
index=main purchase
```

Task 2.3: Add the Source Type

```
index=main sourcetype=access_combined_wcookie purchase
```

✓ **Checkpoint**: All three return purchase events, but each added filter lets Splunk eliminate more data up front. Compare how long each takes to complete.

Exercise 3: Inclusion Beats Exclusion

Overview

Telling Splunk what you *want* is cheaper than telling it what you do not want. `NOT` and `!=` must evaluate every event before discarding any.

Task 3.1: Filter by Exclusion

```
index=main sourcetype=access_combined_wcookie status!=200
| stats count
```

Task 3.2: Filter by Inclusion

```
index=main sourcetype=access_combined_wcookie (status=403 OR status=404 OR status=500)
| stats count
```

✓ **Checkpoint**: Both return the same number of events. The second states which values it wants, so Splunk can use the index rather than testing every event.

Task 3.3: Confirm the Status Values

```
index=main sourcetype=access_combined_wcookie
| stats count by status
```

✓ **Checkpoint**: Record the count for each status code. This is how you know the inclusion list in Task 3.2 is complete.

Exercise 4: Filter Early in the Pipeline

Overview

Each command in a pipeline hands its results to the next one. Filtering after a transforming command means the transform did work it did not need to do.

Task 4.1: Filter Late

```
index=main sourcetype=access_combined_wcookie
| stats count by file, status
| search status=404
```

Here `stats` aggregates every event, and only then does `search` discard most of the rows.

Task 4.2: Filter Early

```
index=main sourcetype=access_combined_wcookie status=404
| stats count by file
```

✓ **Checkpoint**: Same information, but the second search filtered before aggregating. Compare the number of rows `stats` had to produce in each case.

Exercise 5: Wildcard Placement

Overview

A wildcard at the end of a term still lets Splunk use its indexes. A wildcard at the beginning does not — Splunk must examine every value.

Task 5.1: Trailing Wildcard

```
index=main sourcetype=access_combined_wcookie file=cart*
| stats count by file
```

Task 5.2: Leading Wildcard

```
index=main sourcetype=access_combined_wcookie file=*.do
| stats count by file
```

✓ **Checkpoint**: Both return results, but the leading wildcard forces a scan of every `file` value. Prefer an exact value or a trailing wildcard where you can.

Task 5.3: Replace the Wildcard Entirely

```
index=main sourcetype=access_combined_wcookie (file=cart.do OR file=success.do)
| stats count by file
```

✓ **Checkpoint**: When you know the values, list them. This is the fastest of the three.

Exercise 6: Reading the Job Inspector

Overview

The Job Inspector shows what a search actually did, rather than what you intended it to do. It is the first place to look when a search is slow.

Task 6.1: Open the Job Inspector

1. Run any search from this lab
2. Click **Job** above the results, then **Inspect Job**

Task 6.2: Read the Key Numbers

In the Job Inspector, find:

- **Events scanned** — how many events Splunk read
- **Events matched** — how many survived your filters
- **Duration** of each processing phase under **Execution costs**

Task 6.3: Compare Two Searches

Run each of these and inspect the job for both:

```
index=main sourcetype=access_combined_wcookie
| search status=404
```

```
index=main sourcetype=access_combined_wcookie status=404
```

✓ **Checkpoint**: Both return the same events. Compare events scanned against events matched for each. A large gap between the two means Splunk read a lot of data it then threw away.

Exercise 7: Troubleshooting Broken Searches

Overview

Each search below is wrong. Work out why, then repair it.

Task 7.1: Returns Nothing

```
index=main sourcetype=access_combined_wcookie Status=200
| stats count
```

Question: Why does this return zero results when Task 3.3 showed plenty of status 200 events?

Task 7.2: Returns Too Much

```
index=main sourcetype=access_combined_wcookie file=success.do OR status=200
| stats count
```

Question: This returns far more than the successful purchases you expected. What is wrong with the logic?

Task 7.3: Returns the Wrong Shape

```
index=main sourcetype=access_combined_wcookie action=purchase
| stats count by productId
| sort count
```

Question: You wanted the best-selling product at the top. Why is the least popular there instead?

✓ **Checkpoint**: Repair all three and confirm each returns what was intended.

Challenge Exercises

Challenge 1: Rewrite for Speed

This search works but is written poorly. Rewrite it so it filters as early and as specifically as possible, then compare Job Inspector figures.

```
index=*
| search sourcetype=access_combined_wcookie
| search *404*
| stats count by file
```

Challenge 2: Find the Expensive Command

```
index=main sourcetype=access_combined_wcookie
| stats count by clientip, file, status, productId, JSESSIONID
```

Run this, inspect the job, and explain why it is costly. Then produce the same business answer — which files are most requested — far more cheaply.

Challenge 3: Narrow Without Losing Rows

Write a search returning only 403 and 500 responses, using inclusion rather than exclusion, and confirm the total matches the sum of those two status counts from Task 3.3.

Challenge 4: Scoped Investigation

The IP `92.46.53.223` is the noisiest client in the dataset. Write the most tightly scoped search you can that returns only its 403 responses, and justify each filter you included.

Lab Summary

What You Accomplished

- Measured the effect of time range, index scope and source type on a search
- Chose inclusion over exclusion and measured that both agree
- Moved a filter ahead of a transforming command
- Placed wildcards so Splunk can still use its indexes
- Used the Job Inspector to compare events scanned against events matched
- Diagnosed three broken searches

Key Concepts Covered

- **Time as a filter**: the cheapest way to eliminate data
- **Search scope**: `index` and `sourcetype` before keywords
- **Inclusion over exclusion**: state what you want
- **Filter early**: transforming commands should receive the smallest possible set
- **Wildcard placement**: trailing is usable, leading is not
- **Case sensitivity**: field *names* are case sensitive, field *values* are not

Next Steps

- Proceed to Lab 6: Transforming Commands

Answer Key

🔑 **Instructor answer key**

Exercise 1

- Last 30 days: **131,645** events — the whole dataset
- All time: **131,645** events — identical, because the dataset spans about 26 days and so sits entirely inside the 30-day window. Same answer, more work: Splunk had to check every bucket to establish it
- Last 24 hours: **a few thousand**, roughly one day out of twenty-six. This figure is deliberately not pinned down. The dataset's timestamps are anchored to the moment `generate_course_data.py` last ran, so a relative window like "Last 24 hours" returns a different count depending on how long ago that was — and **zero** if the data was regenerated more than a day ago, because the newest event then falls outside the window entirely.

  That sensitivity is the exercise. A relative time range is evaluated against the clock, not against the data, which is exactly why the time picker is the first thing to check when a search returns nothing.

  > **Instructor note**: regenerate the data the evening before or the morning of the class if you want this task to return a non-zero count.

Exercise 3

- Task 3.1 (`status!=200`): **24,402**
- Task 3.2 (inclusion list): **24,402** — identical
- Task 3.3 status counts:

status	count
200	107,243
403	1,936
404	20,477
500	1,989

Exercise 4

Both tasks report the same 404 counts by file. Task 4.1 makes `stats` build a row for every file and status combination before discarding most of them; Task 4.2 gives `stats` only the 404 events to begin with.

Exercise 5

- `file=cart*` matches **cart.do**
- `file=*.do` matches **cart.do** and **success.do**
- The explicit `OR` list in Task 5.3 returns the same rows as Task 5.2 without scanning every value

Exercise 7

- **Task 7.1**: Returns **0**. Field *names* are case sensitive. `Status` is not a field in this data; the field is `status`. Fix: `status=200`
- **Task 7.2**: Returns **107,403** instead of the intended **3,054**. The `OR` reads as "file is success.do, *or* status is 200", which matches every successful request of any kind. Fix: `file=success.do status=200` (terms are ANDed implicitly), or parenthesise explicitly
- **Task 7.3**: `sort count` sorts ascending, so DC-SG-G02 (**184**) appears first instead of WSC-MG-G10. Fix: `sort -count`

Challenge 1

```
index=main sourcetype=access_combined_wcookie status=404
| stats count by file
```

`index=*` searches every index; the `*404*` term forces a full scan and would also match an unrelated value containing 404. Naming the index and filtering on the `status` field removes both problems.

Challenge 2

Grouping by five fields — including `JSESSIONID`, which is nearly unique per session — produces close to one row per event, so `stats` does a great deal of work to return almost no aggregation. The cheap equivalent:

```
index=main sourcetype=access_combined_wcookie
| stats count by file
```

Challenge 3

```
index=main sourcetype=access_combined_wcookie (status=403 OR status=500)
| stats count
```

Total: **3,925** (1,936 + 1,989)

Challenge 4

```
index=main sourcetype=access_combined_wcookie clientip=92.46.53.223 status=403
```

Returns **350** events. Each filter does distinct work: `index` and `sourcetype` scope the data, `clientip` uses an indexed field, and `status` is an inclusion filter on an exact value.